Looking For Group Synchronization

Gideon Chua

The Problem

- A queue of tanks, healers and DPS, all arriving at the same time
- A party is 1 tank, 1 healer, 3 DPS
- At most n dungeon instances can be active at once
- Each run takes a random time between t_1 and t_2 seconds
- No deadlock, no starvation
- Print parties entering/leaving, instance status, and a summary at the end

Design

- **One thread per instance.** Each instance forms a party, runs the dungeon, repeats.
- **One signaler thread.** Decides which empty instance gets the next party.
- **One mutex** (`mtx`) guards everything shared: the three queue counts, the instance status list, and the turn variables.
- **Two condition variables:**
  - `cv`: signaler ⟶ instances (“your turn”)
  - `cv2`: instances ⟶ signaler (“I took my turn” / “I finished a run”)

Shared State

```
std::mutex mtx;
std::condition_variable cv, cv2;
bool ready = false;   // a turn has been handed out
bool done = false;    // no more full parties possible
int num_working = 0;  // instances currently in a run
int target_id = -2;   // whose turn it is (-1 = everyone)
```

Instance: Waiting for a Turn

```
std::unique_lock<std::mutex> lock(mtx);
if (done) { ready = false; cv2.notify_all(); break; }

cv.wait(lock, [&] {
    return (ready && target_id == id) || target_id == -1;
});
ready = false;   // consume the turn
```

Each instance sleeps until the signaler names it. No busy waiting.

Instance: Forming a Party

```
if (num_dps >= 3 && num_tank >= 1 && num_healer >= 1) {
    num_dps -= 3; num_tank -= 1; num_healer -= 1;
    ++num_working;
    working_list[id] = 1;
    run_time = distrib(gen);   // t1..t2
} else {
    isWorking = false;         // queue can't fill a party
}
```

All five players are taken in one step, under the lock.

Instance: Running the Dungeon

```
}   // lock released here
cv2.notify_all();
std::this_thread::sleep_for(std::chrono::seconds(run_time));
{
    std::unique_lock<std::mutex> lock(mtx);
    working_list[id] = 0;
    ++parties_served[id];
    time_served[id] += run_time;
    --num_working;
    cv2.notify_all();   // tell the signaler a slot opened
}
```

The run happens **outside** the lock, so other instances keep going.

Signaler: Picking an Instance

```
if (num_working == instances)
    cv2.wait(lock, [&] { return num_working < instances; });

if (!(num_dps >= 3 && num_tank >= 1 && num_healer >= 1)) {
    target_id = -1;   // wake everyone so they can exit
    done = true;
} else {
    // empty instance that has waited the longest
    target_id = *std::ranges::max_element(ids, {}, [&](int i) {
        return working_list[i] == 1 ? -1 : patience_list[i];
    });
    patience_list[target_id] = 0;
    // every other empty instance: patience_list[i]++
}
ready = true;
```

Signaler: Handshake

```
cv.notify_all();   // hand out the turn
{
    std::unique_lock<std::mutex> lock(mtx);
    cv2.wait(lock, [&] { return !ready; });
}
```

The signaler waits until the chosen instance has taken its turn before picking again. Only one turn is ever outstanding.

Possible Deadlocks

- **Partial parties.** If roles were taken one at a time (e.g. a lock per role), two instances could each hold a tank and wait on DPS the other one has. Neither finishes.
- **Sleeping while holding the lock.** If an instance held `mtx` during its run, nobody else could form a party or even finish a run.
- **Lost wakeups.** If the signaler notified before an instance started waiting, the instance would sleep forever.
- **Stuck at the end.** When the queue can't form another party, instances waiting on `cv` would never be woken, and `join()` would hang.

How They Are Avoided

- **One mutex, one atomic grab.** All 5 players are checked and removed together under `mtx`. With a single lock there is no circular wait.
- **Lock released before the run.** `sleep_for` happens outside the critical section.
- **Predicate waits.** `cv.wait(lock, pred)` checks the condition under the lock first, so a notify that came early is not lost. It also handles spurious wakeups.
- **Shutdown broadcast.** `target_id = -1` and `done = true` wake every instance, each one sees the queue can't fill a party and exits.

Possible Starvation

- **Instance starvation.** With a plain `notify_all` and a race for the mutex, the same instance could win every time while others sit empty.
- **Signaler spinning.** If the signaler kept looping while every instance was busy, it would burn CPU and compete for the lock with instances trying to finish.
- **Players.** Everyone arrives at the same time and only whole parties can be formed, so the only players left waiting are the leftovers that can’t make a party. These are reported in the summary.

How They Are Avoided

- **Patience counters.** Every time an empty instance is passed over, its patience goes up. The signaler always picks the empty instance with the highest patience, then resets it to 0. Every empty instance gets picked eventually.
- **Targeted turns.** `target_id` names one instance, so only that one proceeds; the rest go back to sleep.
- **Signaler sleeps when full.** If `num_working == instances`, it waits on `cv2` until a run finishes.

Sample Run

```
Instances: 2 | Queue: 3 tank, 3 healer, 10 dps | Clear time: 1-2s
Full parties that can be formed: 3

[t=  0s] Party #1 entered instance 0 (clear time: 2s) | queue left: 2 tank, 2 healer, 7 dps
  Instance status:  [0] active  [1] empty
[t=  0s] Party #2 entered instance 1 (clear time: 2s) | queue left: 1 tank, 1 healer, 4 dps
  Instance status:  [0] active  [1] active
[t=  2s] Party #1 cleared instance 0 after 2s
  Instance status:  [0] empty   [1] active
[t=  2s] Party #2 cleared instance 1 after 2s
  Instance status:  [0] empty   [1] empty
[t=  2s] Party #3 entered instance 0 (clear time: 2s) | queue left: 0 tank, 0 healer, 1 dps
  Instance status:  [0] active  [1] empty
[t=  4s] Party #3 cleared instance 0 after 2s
  Instance status:  [0] empty   [1] empty

=== Summary ===
Instance    Parties served  Total time served
0                        2                4s
1                        1                2s
Total                    3                6s

Players left in queue: 0 tank, 0 healer, 1 dps
```

Input Validation

- `config.txt` holds `key value` lines: `instances`, `num_tank`, `num_healer`, `num_dps`, `t1`, `t2`
- Rejects unknown, duplicate or missing keys, and values that aren't whole numbers
- `instances` between 1 and 1000, counts not negative, $0 \leq t_1 \leq t_2 \leq 15$
- Every error is reported with its line number, not just the first one
- Not enough players for a party: the program says so and exits cleanly

Build and Run

```
make
./main                  # reads config.txt
./main other.txt        # or a different config
```

C++20, `g++`, `-pthread`. Built with ThreadSanitizer (`-fsanitize=thread`) during development.

Takeaways

- **One lock is enough** when the critical section is small. It also rules out circular waits.
- **Grab everything at once.** Taking a full party in one step avoids hold-and-wait.
- **Never sleep holding a lock.**
- **Condition variables need a predicate** to survive early and spurious wakeups.
- **Fairness needs a policy.** Without patience counters, the mutex decides who runs.
- **Plan the shutdown.** Waiting threads need to be woken so they can exit.